

NAME : THAMARASELVI S

REG NO : 192511042

COURSE : DESIGN AND ANALYSIS OF ALGORITHMS

CODE : CSA 0617

TOPIC 1 : INTRODUCTION

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1:

Input: words = ["abc","car","ada","racecar","cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada".

Note that "racecar" is also palindromic, but it is not the first.

Example 2:

Input: words = ["notapalindrome","racecar"]

Output: "racecar"

Explanation: The first and only string that is palindromic is "racecar".



main.py

Visualize

Run ▶

Output



JS

TS

```
1 words = ["abc", "car", "ada", "racecar"]
2 for word in words:
3     if word == word[::-1]:
4         print(word)
5         break
6 else:
7     print("")
```

ada

2. You are given two integer arrays `nums1` and `nums2` of sizes `n` and `m`, respectively. Calculate the following values: `answer1` : the number of indices `i` such that `nums1[i]` exists in `nums2`. `answer2` : the number of indices `i` such that `nums2[i]` exists in `nums1` Return `[answer1,answer2]`.

Example 1:

Input: `nums1 = [2,3,2]`, `nums2 = [1,2]`

Output: `[2,1]`

Explanation:

Example 2:

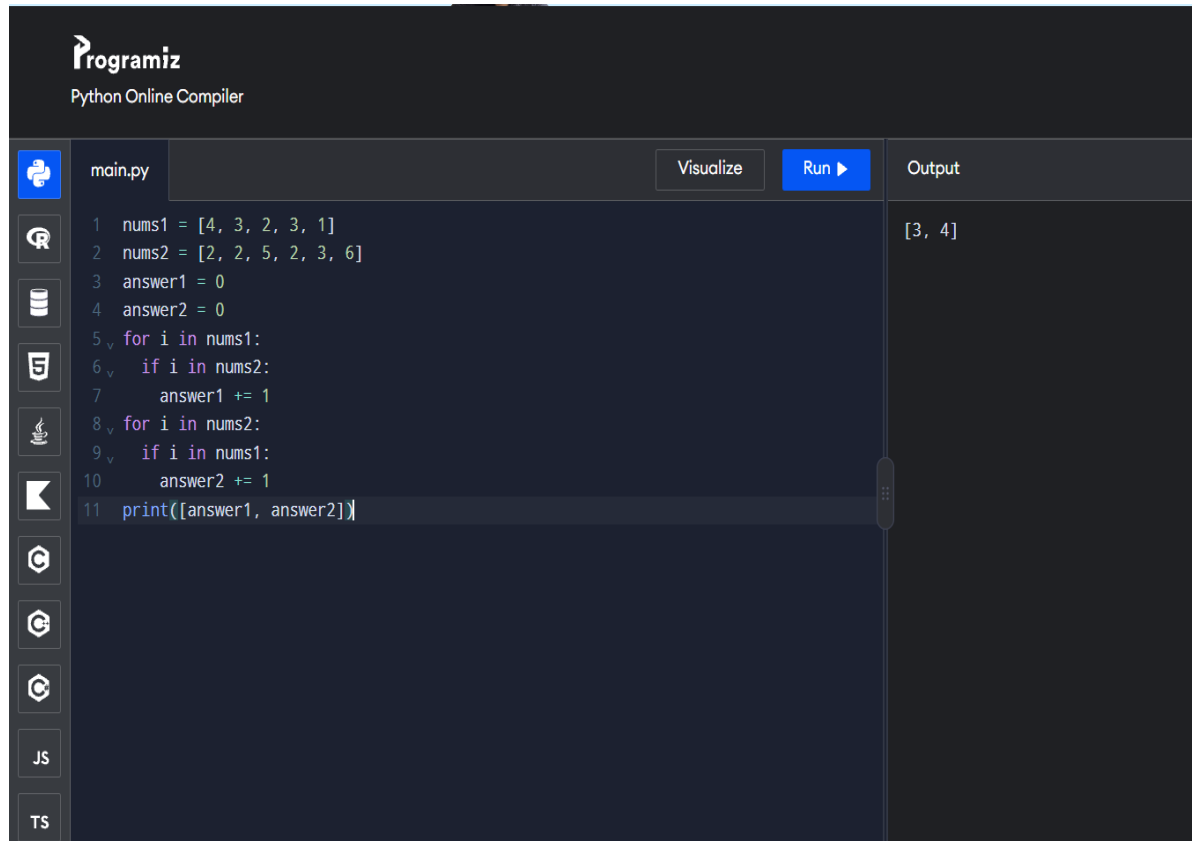
Input: `nums1 = [4,3,2,3,1]`, `nums2 = [2,2,5,2,3,6]`

Output: `[3,4]`

Explanation:

The elements at indices 1, 2, and 3 in nums1 exist in nums2 as well. So answer1 is 3.

The elements at indices 0, 1, 3, and 4 in nums2 exist in nums1. So answer2 is 4.



The screenshot shows the Programiz Python Online Compiler interface. The main editor displays a Python script named 'main.py' with the following code:

```
1 nums1 = [4, 3, 2, 3, 1]
2 nums2 = [2, 2, 5, 2, 3, 6]
3 answer1 = 0
4 answer2 = 0
5 for i in nums1:
6     if i in nums2:
7         answer1 += 1
8 for i in nums2:
9     if i in nums1:
10        answer2 += 1
11 print([answer1, answer2])
```

The 'Output' panel on the right shows the result of the execution: [3, 4]. The interface includes a 'Visualize' button and a 'Run' button (highlighted in blue). A sidebar on the left contains icons for various programming languages, with Python selected at the top.

3. You are given a 0-indexed integer array `nums`. The distinct count of a subarray of `nums` is defined as: Let `nums[i..j]` be a subarray of `nums` consisting of all the indices from `i` to `j` such that $0 \leq i \leq j < \text{nums.length}$. Then the number of distinct values in `nums[i..j]` is called the distinct count of `nums[i..j]`. Return the sum of the squares of distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,2,1]`

Output: 15

Explanation: Six possible subarrays are:

[1]: 1 distinct value

[2]: 1 distinct value

[1]: 1 distinct value

[1,2]: 2 distinct values

[2,1]: 2 distinct values

[1,2,1]: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 + 2^2 + 2^2 + 2^2 = 15$.

Example 2:

Input: nums = [1,1]

Output: 3

Explanation: Three possible subarrays are:

[1]: 1 distinct value

[1]: 1 distinct value

[1,1]: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 = 3$.

```
1  nums = [1, 2, 1]
2
3  total = 0
4
5  for i in range(len(nums)):
6      distinct = set()
7
8      for j in range(i, len(nums)):
9          distinct.add(nums[j])
10         count = len(distinct)
11         total += count * count
12
13 print(total)
```

Output: 15

4. Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return *the number of pairs* (i, j) *where* $0 \leq i < j < n$, *such that* `nums[i] == nums[j]` *and* $(i * j)$ *is divisible by* `k`.

Example 1:

Input: `nums = [3,1,2,2,2,1,3]`, `k = 2`

Output: 4

Explanation:

There are 4 pairs that meet all the requirements:

- `nums[0] == nums[6]`, and $0 * 6 == 0$, which is divisible by 2.
- `nums[2] == nums[3]`, and $2 * 3 == 6$, which is divisible by 2.

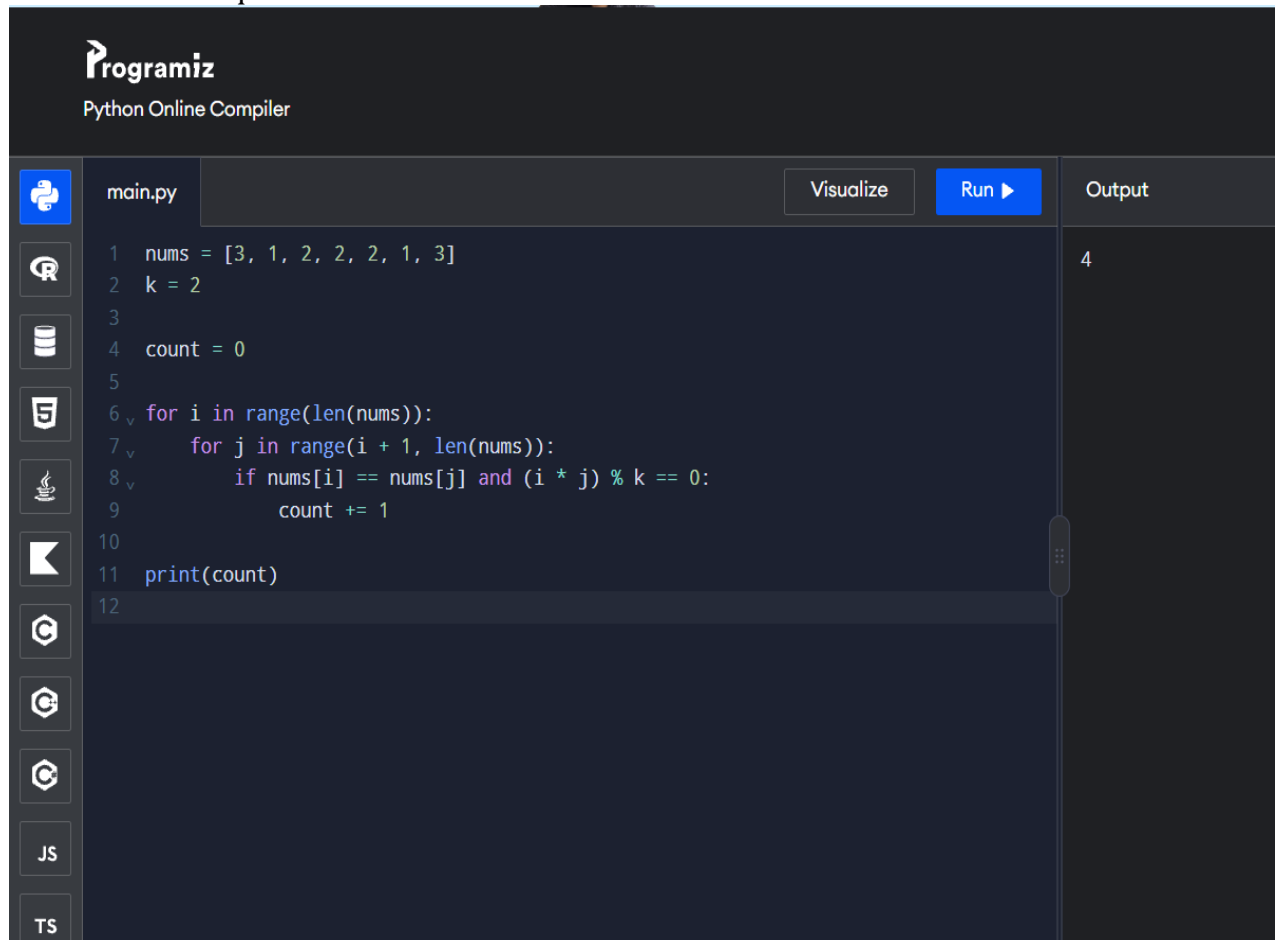
- `nums[2] == nums[4]`, and `2 * 4 == 8`, which is divisible by 2.
- `nums[3] == nums[4]`, and `3 * 4 == 12`, which is divisible by 2.

Example 2:

Input: `nums = [1,2,3,4]`, `k = 1`

Output: 0

Explanation: Since no value in `nums` is repeated, there are no pairs (i,j) that meet all the requirements.



The screenshot shows the Programiz Python Online Compiler interface. The editor contains a Python script named `main.py` with the following code:

```
1  nums = [3, 1, 2, 2, 2, 1, 3]
2  k = 2
3
4  count = 0
5
6  for i in range(len(nums)):
7      for j in range(i + 1, len(nums)):
8          if nums[i] == nums[j] and (i * j) % k == 0:
9              count += 1
10
11 print(count)
12
```

The `Visualize` and `Run` buttons are visible above the code editor. The `Output` panel on the right displays the result of the execution:

```
4
```

5. Write a program FOR THE BELOW TEST CASES with least time complexity
Test Cases: -

Input: `{1, 2, 3, 4, 5}` Expected Output: 5

Input: `{7, 7, 7, 7, 7}` Expected Output: 7

Input: `{-10, 2, 3, -4, 5}` Expected Output: 5

Programiz
Python Online Compiler

main.py Visualize Run ▶ Output

```
1 def find_max(nums):  
2     maximum = nums[0]  
3  
4     for num in nums:  
5         if num > maximum:  
6             maximum = num  
7  
8     return maximum  
9  
10  
11 # Test cases  
12 print(find_max([1, 2, 3, 4, 5]))  
13 print(find_max([7, 7, 7, 7, 7]))  
14 print(find_max([-10, 2, 3, -4, 5]))
```

5
7
5

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

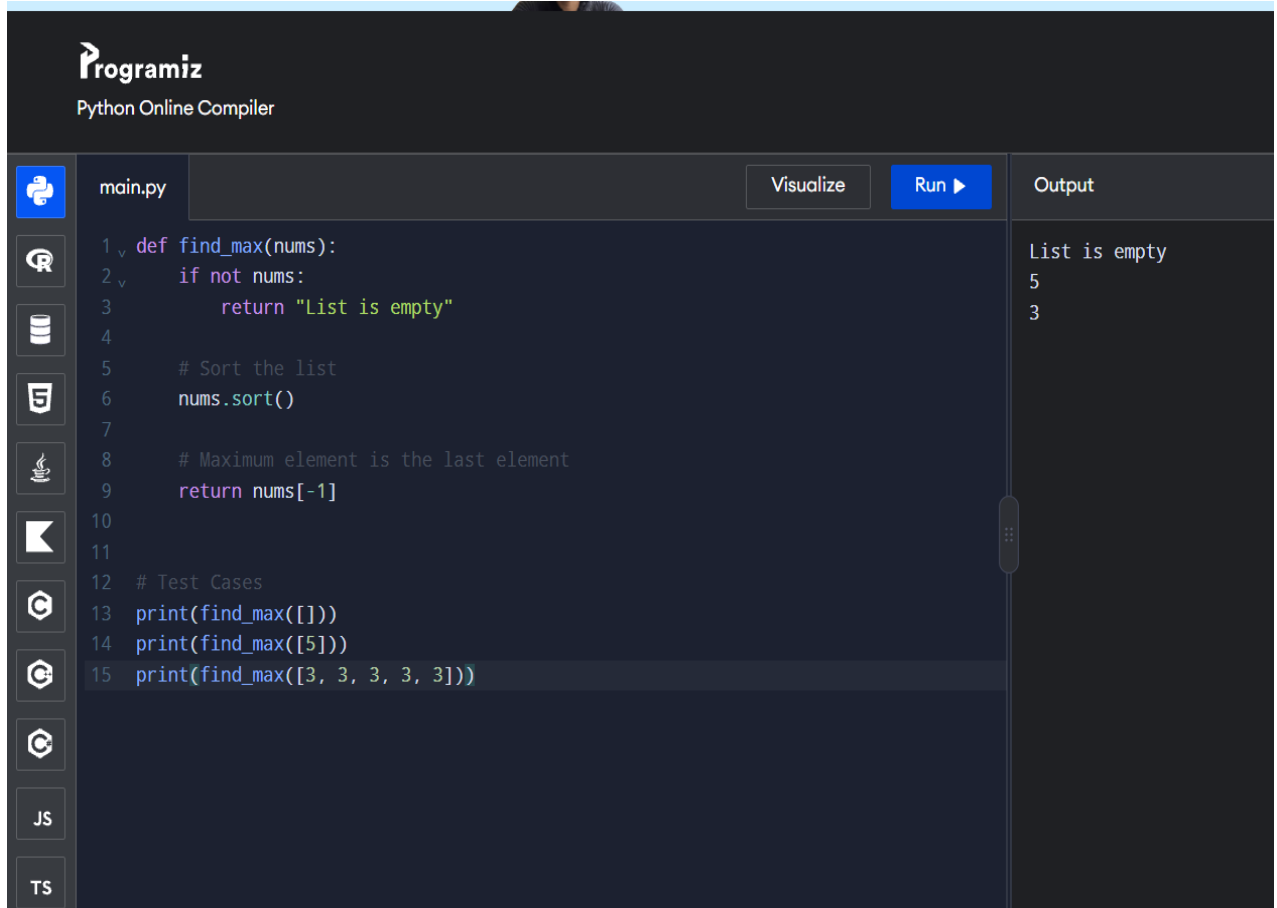
Test Cases

1. Empty List

1. Input: []
2. Expected Output: None or an appropriate message indicating that the list is empty.

2. Single Element List

1. Input: [5]
2. Expected Output: 5
3. All Elements are the Same
 1. Input: [3, 3, 3, 3, 3]
 2. Expected Output: 3



The screenshot shows the Programiz Python Online Compiler interface. The code editor contains the following Python code:

```
1 def find_max(nums):  
2     if not nums:  
3         return "List is empty"  
4  
5     # Sort the list  
6     nums.sort()  
7  
8     # Maximum element is the last element  
9     return nums[-1]  
10  
11  
12 # Test Cases  
13 print(find_max([]))  
14 print(find_max([5]))  
15 print(find_max([3, 3, 3, 3, 3]))
```

The output panel on the right shows the results of the test cases:

```
List is empty  
5  
3
```

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

Input: [3, 7, 3, 5, 2, 5, 9, 2]

Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

Input: [-1, 2, -1, 3, 2, -2]

Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

Input: [1000000, 999999, 1000000]

Expected Output: [1000000, 999999]

Programiz
Python Online Compiler

main.py Visualize Run ▶

```
1 def unique_elements(nums):
2     unique = []
3
4     for num in nums:
5         if num not in unique:
6             unique.append(num)
7
8     return unique
9
10
11 # Test Cases
12 print(unique_elements([3, 7, 3, 5, 2, 5, 9, 2]))
13 print(unique_elements([-1, 2, -1, 3, 2, -2]))
14 print(unique_elements([1000000, 999999, 1000000]))
```

Output

```
[3, 7, 5, 2, 9]
[-1, 2, 3, -2]
[1000000, 999999]
```

JS TS

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

The screenshot shows the Programiz Python Online Compiler interface. The main editor displays a Python script for a bubble sort function. The script is as follows:

```
1 def bubble_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         for j in range(0, n - i - 1):
6             if arr[j] > arr[j + 1]:
7                 # Swap
8                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
9
10    return arr
11
12
13 # Test case
14 arr = [5, 3, 8, 4, 2]
15
16 print("Original array:", arr)
17 print("Sorted array:", bubble_sort(arr))
```

The right-hand side of the interface shows the output of the program:

```
Original array: [5, 3, 8, 4, 2]
Sorted array: [2, 3, 4, 5, 8]
```

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Test Case:

Example $X = \{ 3, 4, 6, -9, 10, 8, 9, 30 \}$ KEY=10

Output: Element 10 is found at position 5

Example $X = \{ 3, 4, 6, -9, 10, 8, 9, 30 \}$ KEY=100

Output : Element 100 is not found

The screenshot shows the Programiz Python Online Compiler interface. The main editor displays a Python script for a binary search function. The script sorts an array and then searches for a key. The output panel shows the result of running the code with the key 10.

```
1 def binary_search(arr, key):
2     arr.sort()
3
4     low = 0
5     high = len(arr) - 1
6
7     while low <= high:
8         mid = (low + high) // 2
9
10        if arr[mid] == key:
11            return mid + 1
12        elif arr[mid] < key:
13            low = mid + 1
14        else:
15            high = mid - 1
16
17    return -1
18
19
20 arr = [3, 4, 6, -9, 10, 8, 9, 30]
21
22 key = int(input("Enter key: "))
```

Output:

```
Enter key: 10
Element 10 is found at position 7
```

10. Given an array of integers `nums`, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

The screenshot shows the Programiz Python Online Compiler interface. The main editor displays a Python script for merge sort. The script defines two functions: `merge_sort` and `merge`. `merge_sort` is a recursive function that splits the array into halves until it reaches a base case of length 1 or less, then merges the sorted halves back together. `merge` is a helper function that merges two sorted arrays into a single sorted array. The output panel on the right shows the result of running the code: "Sorted array: [1, 2, 3, 5, 8]".

```
1 def merge_sort(nums):
2     if len(nums) <= 1:
3         return nums
4
5     mid = len(nums) // 2
6
7     left = merge_sort(nums[:mid])
8     right = merge_sort(nums[mid:])
9
10    return merge(left, right)
11
12
13 def merge(left, right):
14     result = []
15     i = 0
16     j = 0
17
18     while i < len(left) and j < len(right):
19         if left[i] <= right[j]:
20             result.append(left[i])
21             i += 1
22         else:
```

11. Given an $m \times n$ grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Example:

- Input: $m=2, n=2, N=2, i=0, j=0$ • Output: 6
- Input: $m=1, n=3, N=3, i=0, j=1$ • Output: 12



main.py

Visualize

Run ▶

Output



```
1 def find_paths(m, n, N, i, j):
2     MOD = 10**9 + 7
3
4     dp = [[0] * n for _ in range(m)]
5     dp[i][j] = 1
6
7     result = 0
8
9     for step in range(N):
10        new_dp = [[0] * n for _ in range(m)]
11
12        for r in range(m):
13            for c in range(n):
14                if dp[r][c] > 0:
15
16                    # Up
17                    if r == 0:
18                        result += dp[r][c]
19                    else:
20                        new_dp[r - 1][c] += dp[r][c]
21
22                # Down
```

6
12

12. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Examples:

Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the police is 3(robbing house 1).

(ii) Input : nums = [1, 2, 3, 1]

Output : The maximum money you can rob without alerting the

Programiz

Python Online Compiler

main.py

Visualize

Run ▶

Output

Python

R

SQL

Java

C++

C

Go

JS

TS

```
1 def rob_line(nums):
2     prev = 0
3     curr = 0
4
5     for money in nums:
6         new_curr = max(curr, prev + money)
7         prev = curr
8         curr = new_curr
9
10    return curr
11
12
13 def rob(nums):
14     n = len(nums)
15
16     if n == 0:
17         return 0
18
19     if n == 1:
20         return nums[0]
21
22     # Case 1: Rob houses from first to second-last
```

3

4

3

police

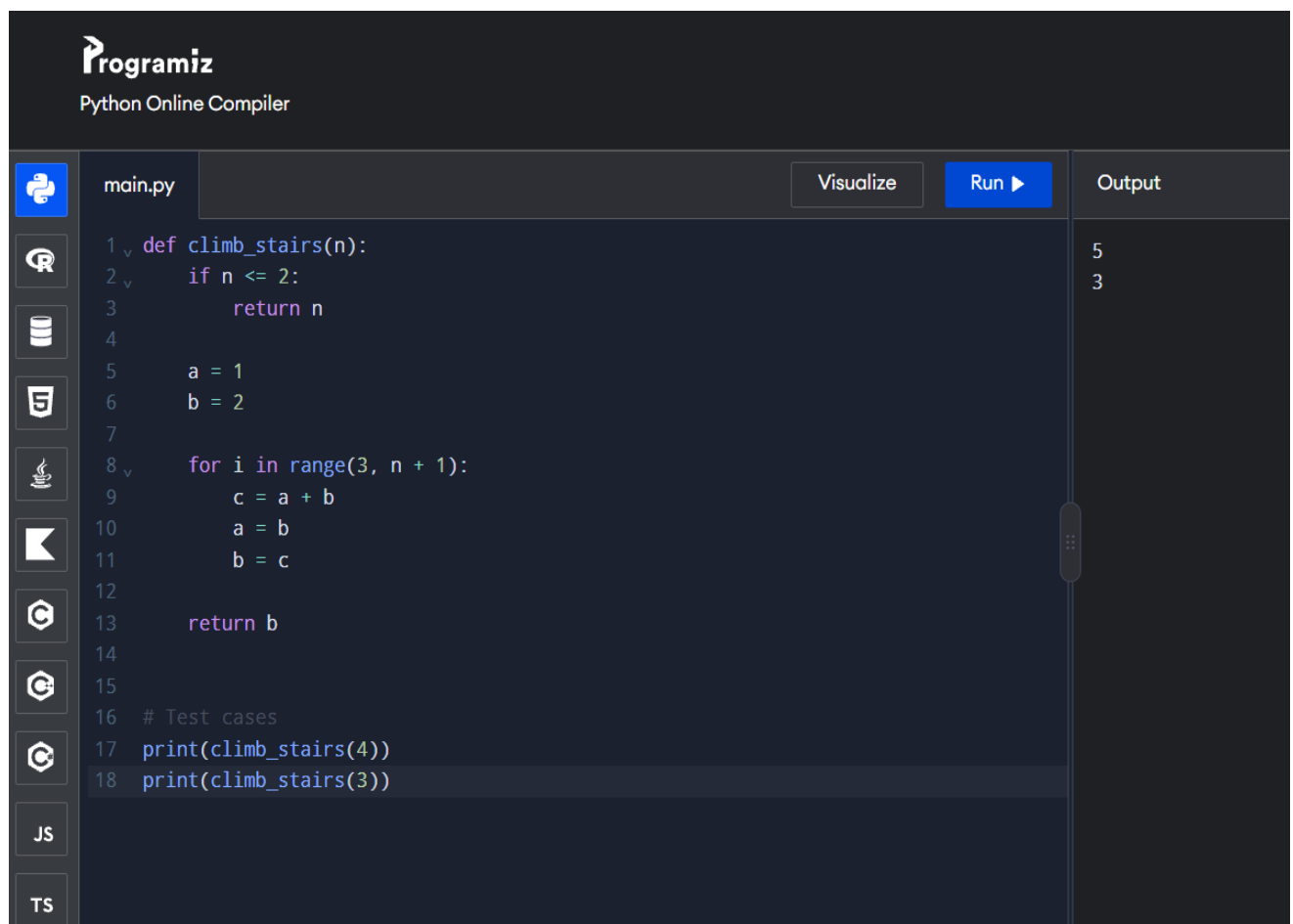
is 4 (robbing house 1 and house 3).

13. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

Input: $n=4$ Output: 5

Input: $n=3$ Output: 3



The screenshot shows the Programiz Python Online Compiler interface. The main editor displays a Python script named `main.py` with the following code:

```
1 def climb_stairs(n):
2     if n <= 2:
3         return n
4
5     a = 1
6     b = 2
7
8     for i in range(3, n + 1):
9         c = a + b
10        a = b
11        b = c
12
13    return b
14
15
16 # Test cases
17 print(climb_stairs(4))
18 print(climb_stairs(3))
```

On the right side of the interface, there are buttons for "Visualize" and "Run". Below these buttons is the "Output" section, which displays the results of the test cases:

```
5
3
```

The left sidebar contains icons for various programming languages and tools, including Python, JavaScript, TypeScript, and others.

14. A robot is located at the top-left corner of a $m \times n$ grid. The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples:

Input: $m=7, n=3$ Output: 28

Input: $m=3, n=2$ Output: 3

Programiz

Python Online Compiler

main.py

Visualize

Run ▶

Output

28

3

```
1 def unique_paths(m, n):
2     dp = [[1] * n for _ in range(m)]
3
4     for i in range(1, m):
5         for j in range(1, n):
6             dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
7
8     return dp[m - 1][n - 1]
9
10
11 # Test cases
12 print(unique_paths(3, 7))
13 print(unique_paths(3, 2))
```

JS

TS

15. In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like $s = \text{"abbxxxxzyy"}$ has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval $[\text{start}, \text{end}]$, where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval $[3, 6]$. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

Example 1:

Input: s = "abbxxxxzzy"

Output: [[3,6]]

Explanation: "xxxx" is the only large group with start index 3 and end index 6.

Example 2:

Input: s = "abc"

Output: []

Explanation: We have groups "a", "b", and "c", none of which are large groups.

```
1 def large_groups(s):
2     result = []
3     start = 0
4
5     for i in range(1, len(s) + 1):
6
7         # End of a group
8         if i == len(s) or s[i] != s[start]:
9
10            # Check group length
11            if i - start >= 3:
12                result.append([start, i - 1])
13
14            start = i
15
16    return result
17
18
19 # Test case
20 s = "abbxxxzyy"
21
22 print(large_groups(s))
```

Output: `[[3, 6]]`

15. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an $m \times n$ grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules

Any live cell with fewer than two live neighbors dies as if caused by under-population.

Any live cell with two or three live neighbors lives on to the next generation.

Any live cell with more than three live neighbors dies, as if by over-population.

Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously. Given the current state of the m x n grid board, return *the next state*.

Example 1:

0	1	0	→	0	0	0
0	0	1		1	0	1
1	1	1		0	1	1
0	0	0		0	1	0

Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]

Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]

Example 2:

1	1	→	1	1
1	0		1	1

Input: board = [[1,1],[1,0]]

Output: [[1,1],[1,1]]

Programiz

Python Online Compiler

main.py

Visualize

Run ▶

Output

[0, 0, 0]

[1, 0, 1]

[0, 1, 1]

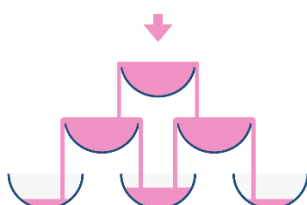
[0, 1, 0]

```

1 def game_of_life(board):
2     rows = len(board)
3     cols = len(board[0])
4
5     new_board = [[0] * cols for _ in range(rows)]
6
7     for i in range(rows):
8         for j in range(cols):
9
10            live_neighbors = 0
11
12            # Check 8 neighbors
13            for x in range(i - 1, i + 2):
14                for y in range(j - 1, j + 2):
15
16                    if x == i and y == j:
17                        continue
18
19                    if 0 <= x < rows and 0 <= y < cols:
20                        live_neighbors += board[x][y]
21
22            # Apply Game of Life rules

```

16. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.



Now after pouring some non-negative integer cups of champagne, return how full the j^{th} glass in the i^{th} row is (both i and j are 0-indexed.)

Example 1:

Input: poured = 1, query_row = 1, query_glass = 1

Output: 0.00000

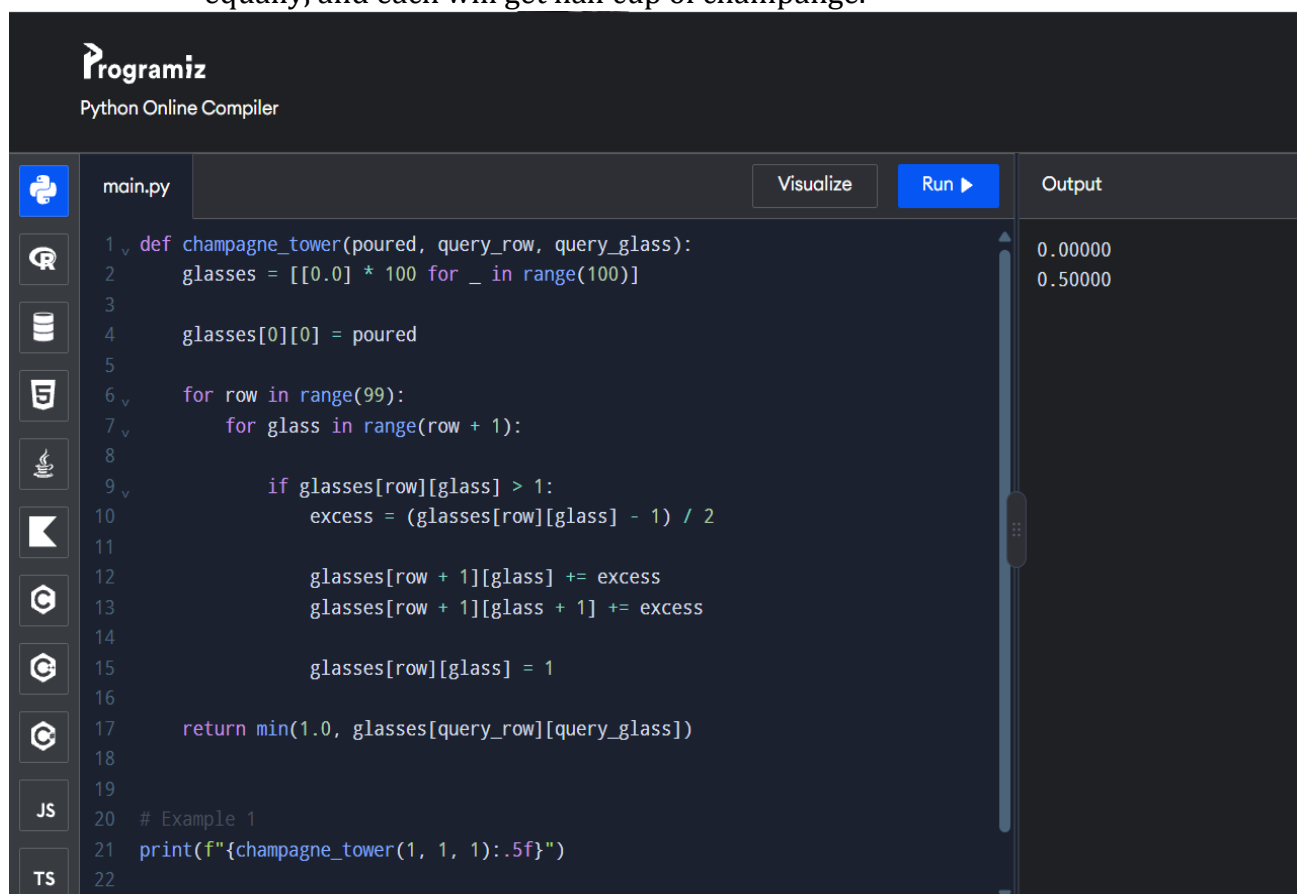
Explanation: We poured 1 cup of champagne to the top glass of the tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty.

Example 2:

Input: poured = 2, query_row = 1, query_glass = 1

Output: 0.50000

Explanation: We poured 2 cups of champagne to the top glass of the tower (which is indexed as (0, 0)). There is one cup of excess liquid. The glass indexed as (1, 0) and the glass indexed as (1, 1) will share the excess liquid equally, and each will get half cup of champagne.



The screenshot shows the Programiz Python Online Compiler interface. The editor contains a Python script for the champagne tower problem. The script defines a function `champagne_tower` that takes `poured`, `query_row`, and `query_glass` as arguments. It initializes a 100x100 grid of glasses, pours the champagne into the top glass (0,0), and then simulates the flow of liquid down the tower. The output of the function is the amount of champagne in the specified glass, capped at 1.0. The example input (1, 1, 1) is used to demonstrate the function.

```
def champagne_tower(poured, query_row, query_glass):
    glasses = [[0.0] * 100 for _ in range(100)]
    glasses[0][0] = poured
    for row in range(99):
        for glass in range(row + 1):
            if glasses[row][glass] > 1:
                excess = (glasses[row][glass] - 1) / 2
                glasses[row + 1][glass] += excess
                glasses[row + 1][glass + 1] += excess
            glasses[row][glass] = 1
    return min(1.0, glasses[query_row][query_glass])

# Example 1
print(f"{champagne_tower(1, 1, 1):.5f}")
```

The output of the program is shown on the right side of the interface, displaying the results for the example input: 0.00000 and 0.50000.